

Operational Tests for Lifecycle Governance (B1.09/B1.10/B1.11): Birth Governance Verification, Mating Governance Verification, Death Governance Verification, Lineage Chain Completeness, and Entity Inventory Reconciliation

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 13, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026).

It does not introduce new axioms. Its sole contribution is to formalize, in deployment-facing operational test form, the lifecycle governance commitments of the CKS architecture — specifically the cluster B1.09/B1.10/B1.11 — so that practitioners can verify whether a given deployment instantiates governed lifecycle primitives or has allowed ungoverned lifecycle dynamics to accumulate.

Abstract

The Coordination Knowledge Substrate (CKS) pattern, as extended in Paper 2 (Li, April 2026), commits to birth, mating, and death as governed primitives that apply uniformly at every level of the architectural hierarchy. These commitments do not enforce themselves: a deployment may allow entities to be created without governance records, combined without mating governance, or removed without archival. Each of these failures produces an anti-pattern — Ungoverned Birth (B3.10), Untracked Mating (B3.12), or Silent Death (B3.11) — that accumulates silently until detected. This note formalizes five deployment-facing tests for the lifecycle governance cluster. Tests 1 through 3 verify each primitive directly: birth governance (B2.44), mating governance (B2.50), and death governance (B2.55). Test 4 verifies structural integrity at the chain level: lineage chain completeness (B2.43). Test 5 performs a population-level arithmetic reconciliation — comparing live entity count against the difference of birth and death event records — whose deviation direction maps directly to a specific anti-pattern. Composite passage requires all five tests to pass. The anti-patterns detectable across the suite are B3.10 Ungoverned Birth, B3.11 Silent Death, B3.12 Untracked Mating, and B3.27 Lifecycle Neglect.

1. Why lifecycle governance requires operational verification

Paper 2's Claim 3 establishes lifecycle primitives — birth, mating, and death — as governed architectural commitments rather than incidental operations. The architecture commits to

governance over each lifecycle decision and to the type-appropriate architectural result of that decision. Birth produces a governed entity with complete specification and lineage anchor. Mating produces an offspring whose DNA combination was performed under an explicitly applied pattern with authorization and parent source records. Death produces either a genuinely deleted entity (functional obsolescence, B2.52) or a retired-but-archived entity (lineage supersession, B2.53) whose substrate content remains addressable.

These commitments are substantive. They require records. They require authorization. They require specific outputs — archival states, lineage chain entries, conflict registry population for union-pattern offspring. A deployment that carries out lifecycle operations without satisfying these requirements is not instantiating the lifecycle governance commitment; it is performing ungoverned lifecycle operations while appearing, at the surface level, to behave similarly.

The problem with ungoverned lifecycle operations is not that they produce immediate failures. They often produce nothing immediately visible. An entity created without a birth record operates normally. An entity removed without a death record simply disappears. A DNA combination performed through direct copy-paste rather than a governed mating event produces an offspring that works. The failures accumulate in the governance layer: the lineage chain develops gaps, the live entity count diverges from the event records, and the deployment loses the retraceability property that makes lifecycle governance valuable in the first place.

The five tests in this note are designed to catch these failures before they compound. They test at three complementary levels: entity-by-entity (Tests 1, 2, 3), chain-structural (Test 4), and population-level accounting (Test 5).

2. Test architecture and source decompositions

The five tests in this note apply the operational test structure established in B5.01: each test states a TEST QUESTION, TEST MECHANISM, PASS CONDITION, FAIL CONDITION, and REMEDIATION SIGNAL. The lifecycle cluster tests draw on the following B2 decompositions:

- B2.43: Lineage chain as retraceable provenance structure with A2.40 metadata at each event
- B2.44: Birth verification — birth record existence, governance authorization, level-appropriate completeness
- B2.50: Mating verification — mating event record, pattern identification, authorization, parent source records, conflict registry
- B2.52–B2.55: Death governance — death record, death pattern identification (obsolescence vs. supersession), archival state, lineage chain closure, membership dissolution

The A2.40 provenance record requirement — six metadata fields per lifecycle event — is the Paper 1 inheritance that underwrites all three lifecycle primitive tests. Without A2.40-compliant records, no lifecycle event can be verified as governed.

3. Test 1 — Birth Governance Test

TEST QUESTION:

Was each active entity in the deployment born through governed origination per B1.09, with a complete birth specification and governance provenance?

TEST MECHANISM (B2.44 birth verification):

For each entity in the deployment: (a) Does a birth record meeting the A2.40 six-field provenance standard exist? (b) Does the birth record include governance authorization per A2.47? (c) Is the birth specification complete at the entity's compositional level — cell-level completeness per B2.99, aspect-level per B2.100, Self-level per B2.101? (d) Does the entity have a lineage anchor per B2.43 that connects subsequent events to this birth record?

PASS CONDITION:

Every active entity has a birth record with A2.40-compliant provenance; every birth record includes governance authorization; every birth specification is complete at the appropriate level; every entity has a lineage anchor. No active entity is unaccounted for by a birth record.

FAIL CONDITION (three sub-forms):

Fail Form 1 — Auto-created entities: Entities exist in the deployment without any birth record. These entities were created through some automated, scripted, or manual process that bypassed governance record creation.

Fail Form 2 — Incomplete specification: Birth records exist but the birth specification is incomplete for the entity's level. The entity was created with governance intent but the specification work was not completed.

Fail Form 3 — Unrecorded birth: Birth records exist for some but not all entities, with the remainder having no trace of governed origination.

REMEDIATION SIGNAL:

Ungoverned Birth (B3.10). Determine the sub-form using the fail condition taxonomy above. Remediate per B3.10 remediation guidance: retroactively construct birth records for auto-created entities where the origination intent can be recovered; complete incomplete specifications; audit creation pipelines for bypass pathways.

4. Test 2 — Mating Governance Test

TEST QUESTION:

Were any entities in the deployment created through DNA combination (mating per B1.10), and if so, were they created through governed mating patterns?

TEST MECHANISM (B2.50 mating verification):

Identify entities whose DNA contains elements traceable to multiple parent entities — this is the structural signature of mating. For each such entity: (a) Does a mating event record per A2.40 exist? (b) Was a mating pattern explicitly applied — Union (B2.46), Selective Merge (B2.47), or Lineage-Preserved Union (B2.48)? (c) Were governance authorization and parent source records created at mating time? (d) For entities created through the Union pattern specifically: were conflicts arising from parent DNA registered in the A1.03 conflict registry, making them first-class substrate state rather than silent merges?

PASS CONDITION:

All entities with multi-parent DNA have governed mating records with A2.40-compliant provenance; each mating record names the pattern applied; governance authorization and parent source records are present; Union-pattern offspring have A1.03 conflict registry entries for any merge-time conflicts.

FAIL CONDITION (two primary forms):

Fail Form 1 — Unrecorded combination: Multi-parent DNA entities exist without mating event records. DNA combination occurred — through copy-paste, direct content transfer, or scripted merging — without any governed mating event being recorded.

Fail Form 2 — Patternless combination: Mating records exist but no pattern was applied. The combination is recorded as having occurred but the governance mechanism — pattern selection, authorization, conflict handling — was omitted.

REMEDIATION SIGNAL:

Untracked Mating (B3.12). Remediate per B3.12 remediation guidance: retroactively document the combination as a governed mating event; identify the pattern that best describes how the combination was actually performed; create parent source records from whatever provenance information is recoverable; register any conflicts that would have been captured under the Union pattern.

5. Test 3 — Death Governance Test**TEST QUESTION:**

Were entity closures in the deployment governed per B1.11, with death pattern identification, archival transition (where appropriate), and lineage closure?

TEST MECHANISM (B2.55 death governance verification):

For any entities that have been closed or are absent from the active deployment: (a) Does a death record per A2.40 exist? (b) Was a death pattern identified — functional obsolescence (B2.52) or lineage supersession (B2.53)? (c) For lineage-supersession deaths specifically: was the entity transitioned to archival state per B2.54 rather than deleted, with its substrate content remaining addressable? (d) Was the lineage chain closed per B2.43, with a death record serving as the chain's terminus? (e) Were memberships in aspects or Selves dissolved per B2.55?

PASS CONDITION:

All closed entities have death records with A2.40-compliant provenance; each death record names the death pattern; lineage-supersession entities are in archival state with addressable substrate content; all lineage chains for dead entities terminate with their death records; all memberships are dissolved.

FAIL CONDITION (three sub-forms):

Fail Form 1 — Deletion without governance: Entities are absent from the deployment with no death record. They were deleted directly, without governance record, death pattern identification, or archival consideration.

Fail Form 2 — Non-archived death: Death records exist but lineage-supersession entities were deleted rather than archived. The substrate content is no longer addressable.

Fail Form 3 — Lineage-unclosed deactivation: Death records exist but lineage chains have no terminus for the absent entity. The chain is incomplete on the death end.

REMEDIATION SIGNAL:

Silent Death (B3.11). Determine the sub-form using the fail condition taxonomy above. For Form 1, reconstruct death records where the closure intent can be recovered and determine the applicable death pattern retroactively. For Form 2, determine whether the deleted content can be recovered; if not, record the non-archival as a governance failure in the death record. For Form 3, close the lineage chains using the available death records.

6. Test 4 — Lineage Chain Completeness Test

TEST QUESTION:

Does each entity have a complete, retraceable lineage chain from birth through its current state — or, for dead entities, from birth through the terminus death record?

TEST MECHANISM (B2.43 lineage chain inspection):

For each entity: (a) Trace from current state back to the birth record; verify that each event in the chain carries A2.40 provenance; verify that the chain has no unexplained gaps — no segment where one event cannot be linked to its predecessor. (b) For mated entities: verify that the lineage chain includes cross-lineage references linking to parent birth records, so the full provenance of multi-parent DNA is retraceable. (c) For dead entities: verify that the chain terminates with a death record rather than terminating at the last live event without closure.

PASS CONDITION:

All lineage chains are complete and retraceable without gaps; all events in every chain carry A2.40-compliant provenance; mated entity chains link to parent chains; dead entity chains terminate with death records.

FAIL CONDITION (four forms):

Chain gaps: One or more events in a chain are unrecorded, producing a segment that cannot be traced.

Birth-anchor absence: A chain has no birth record at its origin; the chain has no anchor.

Death-terminus absence: A dead entity's chain terminates at its last live event without a death record.

Parent-reference absence: A mated entity's chain has no cross-lineage references; the multi-parent provenance is untraceable.

REMEDIATION SIGNAL:

Provenance Void (B3.23) if the gap pattern is systematic across multiple chains, indicating a deployment-wide failure to maintain chain integrity rather than isolated incidents. For individual gaps: remediate per the lifecycle event type whose absence produced the gap — missing birth record (B3.10 remediation), missing death record (B3.11 remediation), missing mating record

(B3.12 remediation), or missing intermediate event (reconstruct from available operational records per A2.40).

7. Test 5 — Entity Inventory Reconciliation Test

TEST QUESTION:

Does the current live entity population reconcile with the deployment's lifecycle event records?

TEST MECHANISM:

This test operates at the population level rather than the entity level, using a single arithmetic identity to cross-validate the entire lifecycle event record against the observed entity population.

Entity count reconciliation: Count live entities in the deployment (L). Count birth events in the lifecycle event records (B). Count death events in the lifecycle event records (D). These should satisfy: $L = B - D$. Any deviation from this identity is a lifecycle governance failure.

Anomaly direction analysis: If $L > B - D$, there are more live entities than the event records account for — ungoverned births have occurred. If $L < B - D$, fewer entities are present than the event records account for, with birth records but no corresponding death records for the absent entities — silent deaths have occurred. Both conditions may be present simultaneously.

Archival verification: For entities present in death records under lineage-supersession death pattern (B2.53): verify that these entities are in archived state and that their substrate content remains addressable. These entities are operationally retired but not absent from the substrate; they should not be counted in L but should be verifiable as archived.

PASS CONDITION:

$L = B - D$; every live entity maps to a birth record; every entity with a birth record that is absent from the live population maps to a death record; all lineage-supersession deaths are in confirmed archival state.

FAIL CONDITION:

$L > B - D$: Ungoverned births are present. Some live entities have no birth records.

Absent entities without death records: Entities with birth records are absent from the live population without corresponding death records. Silent deaths are present.

Both conditions simultaneously: The lifecycle event record is in significant disorder; both ungoverned creations and ungoverned deletions have occurred.

REMEDIATION SIGNAL:

The deviation direction determines the anti-pattern: if $L > B - D$, the primary signal is Ungoverned Birth (B3.10); if absent entities lack death records, the primary signal is Silent Death (B3.11); if both conditions are present, the composite signal is Lifecycle Neglect (B3.27). Lifecycle Neglect names the condition in which lifecycle governance has been abandoned broadly rather than violated in isolated cases — both creation and retirement are ungoverned, and the event record has lost integrity as an accounting of the entity population.

The entity inventory reconciliation test is particularly powerful as a deployment-level diagnostic because it requires no entity-by-entity traversal to identify a governance problem: the arithmetic identity either holds or it does not, the direction of deviation is immediately interpretable, and the anti-pattern designation follows directly from the deviation type. Tests 1 through 4 are required to identify which specific entities have governance failures; Test 5 is required to determine whether lifecycle governance is functioning at the population level at all.

8. Composite lifecycle test result

COMPOSITE PASSAGE CONDITION:

All five tests must pass for a deployment to achieve full lifecycle governance operational test passage.

A deployment that passes Tests 1, 2, and 3 but fails Test 4 has individual lifecycle events recorded but has not maintained chain integrity — the events exist in isolation rather than as a retraceable lineage. A deployment that passes Tests 1 through 4 but fails Test 5 has individual entity records and chains that appear intact but has an entity population that does not reconcile with the event records — a population-level failure invisible to entity-level inspection alone.

Anti-patterns detectable by this suite:

Anti-pattern	Detected by	Fail condition
B3.10 Ungoverned Birth	Tests 1, 5	Active entities without birth records; $L > B - D$
B3.11 Silent Death	Tests 3, 5	Absent entities without death records; chains without terminus
B3.12 Untracked Mating	Test 2	Multi-parent DNA entities without mating records
B3.23 Provenance Void	Test 4	Systematic gaps across lineage chains
B3.27 Lifecycle Neglect	Test 5 (composite)	Both B3.10 and B3.11 present simultaneously

Test execution order:

Tests 1 through 3 may be run in any order relative to each other. Test 4 depends on the records surfaced by Tests 1 through 3 — chain completeness can only be assessed once all lifecycle event records have been identified as present or absent. Test 5 is most usefully run as the final check: it provides the population-level validation that confirms the individual findings are not isolated anomalies but reflect (or do not reflect) systematic lifecycle governance.

9. Conclusion

Lifecycle governance in a CKS deployment is only as strong as the governance records it produces and maintains. The five tests formalized in this note — Birth Governance, Mating Governance, Death Governance, Lineage Chain Completeness, and Entity Inventory Reconciliation — provide a

complete operational test suite for the lifecycle primitive cluster. Tests 1 through 3 verify each primitive at the event level. Test 4 verifies that events compose into complete retraceable chains. Test 5 verifies that the chains collectively account for the live entity population.

The entity inventory reconciliation test is the uniquely diagnostic element of this suite: its arithmetic identity operates as a population-level oracle, and the direction of any deviation — excess live entities or absent entities without death records — maps directly to the applicable anti-pattern designation. A deployment in which all five tests pass has instantiated lifecycle governance as a functioning architectural commitment, not merely as a stated intention.

Source papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Tests for Lifecycle Governance (B1.09/B1.10/B1.11): Birth Governance Verification, Mating Governance Verification, Death Governance Verification, Lineage Chain Completeness, and Entity Inventory Reconciliation*. 13 May 2026. ORCID: 0009-0004-8065-3235.